

分析 Google Play 帳款服務中的產品購買交易放棄率

來源：<https://codelabs.developers.google.com/play-billing-analyze-product-purchase-drop-offs?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將瞭解如何處理一次性產品的購買流程中斷情況。

目錄

1. [簡介](#)
2. [建構範例應用程式](#)
3. [在 Play 管理中心建立單次產品](#)
4. [與 PBL 整合](#)
5. [分析購買交易放棄率](#)
6. [後續步驟](#)
7. [恭喜！](#)

1. 簡介

在本程式碼研究室中，您將著重於建立單次產品、整合應用程式與 Play Billing Library (PBL)，以及分析購買交易中斷的原因。

注意：如要順利完成本程式碼研究室，您必須有權使用「單次產品的多個購買選項和優惠」功能。這項功能目前僅適用於搶先體驗計畫 (EAP)。搶先體驗計畫中的產品和功能是按照「原樣」提供，支援範圍可能有限。如要使用 EAP 功能，請填寫[單次產品 EAP 意願調查表](#)。不過，如果只想瞭解如何使用 Play 帳款服務回應碼分析購買交易放棄率，請直接前往本程式碼研究室的「[分析購買交易放棄率](#)」一節。

觀眾

這個程式碼研究室適用於使用 Play 帳款服務程式庫 (PBL) 的 Android 應用程式開發人員，或想使用 PBL 透過一次性產品營利的開發人員。

課程內容...

- 如何在 Google Play 管理中心建立一次性產品。
- 如何將應用程式與 PBL 整合。
- 瞭解如何在 PBL 中處理消耗性和非消耗性單次產品購買交易。
- 如何分析購物流程中途放棄情形。

事前準備

- 使用開發人員帳戶存取 [Google Play 管理中心](#)。如果沒有開發人員帳戶，請[建立帳戶](#)。
 - 本程式碼研究室的範例應用程式，您可以[從 GitHub 下載](#)。
 - [Android Studio](#)。
-

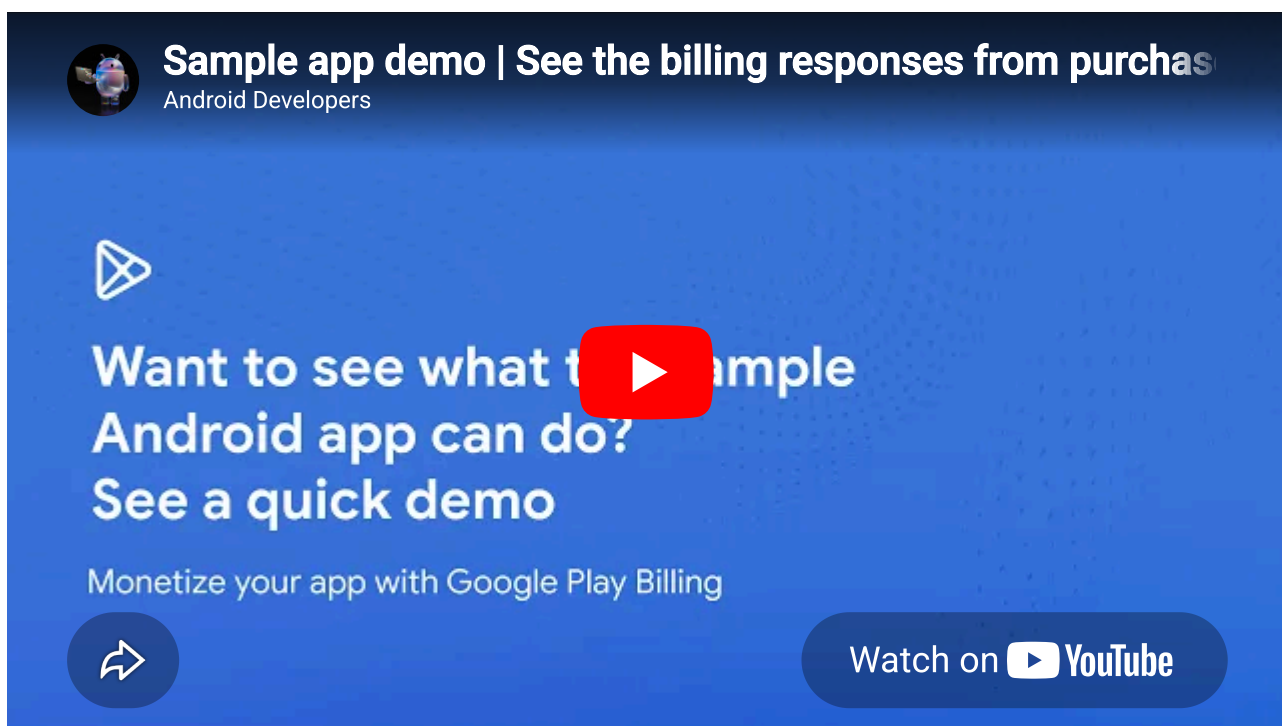
步驟 2 · 約 15 分鐘

2. 建構範例應用程式

這個範例應用程式是功能齊全的 Android 應用程式，提供完整原始碼，可展示下列各方面：

- 將應用程式與 PBL 整合
- 擷取一次性產品
- 啟動一次性產品的購買流程
- 購買情境會導致下列帳單回應：
 - `BILLING_UNAVAILABLE`
 - `USER_CANCELLED`
 - `OK`
 - `ITEM_ALREADY_OWNED`

以下示範影片顯示範例應用程式部署及執行後的樣貌和行為。



必備條件

建構及部署範例應用程式前，請先完成下列步驟：

- [建立 Google Play 管理中心開發人員帳戶](#)。如果已有開發人員帳戶，請略過這個步驟。
- [在 Play 管理中心建立新應用程式](#)。建立應用程式時，您可以為範例應用程式指定任何應用程式名稱。
- 安裝 [Android Studio](#)。

建構

這個建構步驟的目標是產生範例應用程式的已簽署 Android App Bundle 檔案。

如要產生 Android 應用程式套件，請按照下列步驟操作：

1. 從 [GitHub](#) 下載範例應用程式。

2. [建構範例應用程式](#)。建構前，請先變更範例應用程式的套件名稱，然後再建構。如果 Play 管理中心中還有其他應用程式的套件，請確保您為範例應用程式提供的套件名稱是專屬的。

注意：建構範例應用程式只會建立 APK 檔案，可用於本機測試。不過，由於您尚未在 Play 管理中心設定產品，因此執行應用程式不會擷取產品和價格，您將在本程式碼研究室中進一步瞭解這項操作。

3. 產生已簽署的 Android App Bundle。

1. [產生上傳金鑰和 KeyStore](#)
2. [使用上傳金鑰簽署應用程式](#)
3. [設定 Play 應用程式簽署功能](#)

接下來，請將 Android 應用程式套件上傳至 Google Play 管理中心。

3. 在 Play 管理中心建立單次產品

如要在 Google Play 管理中心建立一次性產品，您必須在 Play 管理中心擁有應用程式。在 Play 管理中心建立應用程式，然後上傳先前建立的已簽署應用程式套件。

建立應用程式

如要建立應用程式，請按照下列步驟操作：

1. 使用開發人員帳戶登入 [Google Play 管理中心](#)。
2. 按一下「建立應用程式」，開啟「建立應用程式」頁面。
3. 輸入應用程式名稱、選取預設語言，以及其他應用程式相關詳細資料。
4. 按一下「建立應用程式」，即可在 Google Play 管理中心建立應用程式。

現在可以上傳範例應用程式的已簽署應用程式套件。

上傳已簽署的應用程式套件

1. 將已簽署的應用程式套件上傳至 [Google Play 管理中心的內部測試群組](#)。上傳後，您才能在 Play 管理中心設定營利相關功能。
2. 依序點選「測試及發布」>「測試」>「內部測試版本」>「建立新版本」。
3. 輸入發布版本名稱，然後上傳已簽署的應用程式套件檔案。
4. 依序點選「下一步」和「儲存並發布」。

現在你可以建立單次性產品。

建立單次產品

如何建立單次產品：

1. 在 [Google Play 管理中心](#) 的左側導覽選單中，依序前往「透過 Google Play 營利」>「產品」>「單次產品」。
2. 按一下「建立單次產品」。
3. 輸入下列產品詳細資料：

- **產品 ID**：輸入專屬產品 ID。輸入 `one_time_product_01`。
- **(選用) 標記**：新增相關標記。
- **名稱**：輸入產品名稱。例如：`Product name`。
- **說明**：輸入產品說明。例如：`Product description`。
- **(選用) 新增圖示圖片**：上傳代表產品的圖示。

注意：在本程式碼研究室中，您可以略過「稅金、法規遵循和計畫」部分的設定。

4. 點選 [下一步]。
5. 新增購買選項並設定區域供應情形。單次產品至少需要一個購買選項，用於定義授權方式、價格和區域供應情形。在本程式碼研究室中，我們將為產品新增標準的「購買」選項。在「購買選項」部分，輸入下列詳細資料：
 - **購買選項 ID**：輸入購買選項 ID。例如：`buy`。
 - **交易類型**：選取「購買」。
 - **(選填) 標籤**：新增這個購買選項專屬的標籤。
 - **(選用) 按一下「進階選項」**，設定進階選項。在本程式碼研究室中，您可以略過進階選項設定。

6. 在「適用情形與定價」部分，依序點選「設定價格」 > 「大量編輯價格」。
7. 選取「國家 / 地區」選項。這會選取所有區域。
8. 按一下「繼續」。系統會開啟對話方塊，供你輸入價格。輸入 10 美元，然後按一下「套用」。
9. 依序按一下「儲存」和「啟用」。系統會建立並啟用購買選項。

以本程式碼研究室為例，請建立 3 項額外的單次產品，並使用下列產品 ID：

- consumable_product_01
- consumable_product_02
- consumable_product_03

範例應用程式已設定為使用這些產品 ID。你可以提供不同的產品 ID，但必須修改範例應用程式，才能使用你提供的產品 ID。

在 [Google Play 管理中心](#) 開啟範例應用程式，然後依序前往「透過 Google Play 營利」 > 「產品」 > 「一次性產品」。然後按一下「建立單次產品」，並重複步驟 3 至 9。

單次產品建立影片

以下影片樣本顯示先前所述的單次產品建立步驟。



4. 與 PBL 整合

接下來，我們將說明如何整合應用程式與 [Play 帳款服務程式庫 \(PBL\)](#)。本節說明整合的高階步驟，並提供各步驟的程式碼片段。您可以參考這些程式碼片段，實作實際的整合功能。

如要整合應用程式與 PBL，請按照下列步驟操作：

1. 將 Play 帳款服務程式庫依附元件新增至範例應用程式。

```
dependencies {  
    val billing_version = "8.0.0"  
  
    implementation("com.android.billingclient:billing-ktx:$billing_version")  
}
```

2. 初始化 [BillingClient](#)。BillingClient 是位於應用程式的用戶端 SDK，可與 Play 帳款服務程式庫通訊。下列程式碼片段說明如何初始化帳單用戶端。

```
protected BillingClient createBillingClient() {  
    return BillingClient.newBuilder(activity)  
        .setListener(purchasesUpdatedListener)  
  
        .enablePendingPurchases(PendingPurchasesParams.newBuilder().enableOneTimeProducts().build()  
        ())  
        .enableAutoServiceReconnection()  
        .build();  
}
```

3. 連線至 Google Play。下列程式碼片段說明如何連線至 Google Play。


```

public void startBillingConnection(ImmutableList<Product> productList) {
    Log.i(TAG, "Product list sent: " + productList);
    Log.i(TAG, "Starting connection");
    billingClient.startConnection(
        new BillingClientStateListener() {
            @Override
            public void onBillingSetupFinished(BillingResult billingResult) {
                if (billingResult.getResponseCode() == BillingResponseCode.OK) {
                    // Query product details to get the product details list.
                    queryProductDetails(productList);
                } else {
                    // BillingClient.enableAutoServiceReconnection() will retry the connection on
                    // transient errors automatically.
                    // We don't need to retry on terminal errors (e.g., BILLING_UNAVAILABLE,
                    // DEVELOPER_ERROR).
                    Log.e(TAG, "Billing connection failed: " + billingResult.getDebugMessage());
                    Log.e(TAG, "Billing response code: " + billingResult.getResponseCode());
                }
            }

            @Override
            public void onBillingServiceDisconnected() {
                Log.e(TAG, "Billing Service connection lost.");
            }
        }
    );
}

```

4. 擷取單次產品詳細資料。將應用程式與 PBL 整合後，您必須將單次產品詳細資料擷取到應用程式中。下列程式碼片段說明如何在應用程式中擷取單次產品詳細資料。

```

private void queryProductDetails(ImmutableList<Product> productList) {
    Log.i(TAG, "Querying products for: " + productList);
    QueryProductDetailsParams queryProductDetailsParams =
        QueryProductDetailsParams.newBuilder().setProductList(productList).build();
    billingClient.queryProductDetailsAsync(
        queryProductDetailsParams,
        new ProductDetailsResponseListener() {
            @Override
            public void onProductDetailsResponse(
                BillingResult billingResult, QueryProductDetailsResult productDetailsResponse)
            {
                // check billingResult
                Log.i(TAG, "Billing result after querying: " + billingResult.getResponseCode());
                // process returned productDetailsList
                Log.i(
                    TAG,
                    "Print unfetched products: " +
                    productDetailsResponse.getUnfetchedProductList());
                setupProductDetailsMap(productDetailsResponse.getProductDetailsList());
                billingServiceClientListener.onProductDetailsFetched(productDetailsMap);
            }
        });
}

```

擷取 `ProductDetails`，會傳回類似以下的回應：

```

{
  "productId": "consumable_product_01",
  "type": "inapp",
  "title": "Shadow Coat (Yolo's Realm | Play Samples)",
  "name": "Shadow Coat",
  "description": "A sleek, obsidian coat for stealth and ambushes",
  "skuDetailsToken": "<---skuDetailsToken--->",
  "oneTimePurchaseOfferDetails": {},
  "oneTimePurchaseOfferDetailsList": [
    {
      "priceAmountMicros": 1990000,
      "priceCurrencyCode": "USD",
      "formattedPrice": "$1.99",
      "offerIdToken": "<---offerIdToken--->",
      "purchaseOptionId": "buy",
      "offerTags": []
    }
  ]
},
{
  "productId": "consumable_product_02",
  "type": "inapp",
  "title": "Emperor Den (Yolo's Realm | Play Samples)",
  "name": "Emperor Den",
  "description": "A fair lair glowing with molten rock and embers",
  "skuDetailsToken": "<---skuDetailsToken--->",
  "oneTimePurchaseOfferDetails": {},
  "oneTimePurchaseOfferDetailsList": [
    {
      "priceAmountMicros": 2990000,
      "priceCurrencyCode": "USD",
      "formattedPrice": "$2.99",
      "offerIdToken": "<---offerIdToken--->",
      "purchaseOptionId": "buy",
      "offerTags": []
    }
  ]
}

```

5. 啟動結帳流程。

```

public void launchBillingFlow(String productId) {
    ProductDetails productDetails = productDetailsMap.get(productId);
    if (productDetails == null) {
        Log.e(
            TAG, "Cannot launch billing flow: ProductDetails not found for productId: " +
            productId);
        billingServiceClientListener.onBillingResponse(
            BillingResponseCode.ITEM_UNAVAILABLE,

BillingResult.newBuilder().setResponseCode(BillingResponseCode.ITEM_UNAVAILABLE).build())
        ;
        return;
    }
    ImmutableList<ProductDetailsParams> productDetailsParamsList =
        ImmutableList.of(
            ProductDetailsParams.newBuilder().setProductDetails(productDetails).build());

    BillingFlowParams billingFlowParams =
        BillingFlowParams.newBuilder()
            .setProductDetailsParamsList(productDetailsParamsList)
            .build();

    billingClient.launchBillingFlow(activity, billingFlowParams);
}

```

6. 偵測及處理購買交易。在這個步驟中，您需要：

1. [驗證購買交易](#)
2. 授予使用者授權
3. 通知使用者
4. 通知 Google 購買程序

其中步驟 a、b 和 c 應在後端完成，因此不在本程式碼研究室的範圍內。下列程式碼片段說明如何通知 Google 消耗型單次產品：

```

private void handlePurchase(Purchase purchase) {
// Step 1: Send the purchase to your secure backend to verify the purchase following
// https://developer.android.com/google/play/billing/security#verify

// Step 2: Update your entitlement storage with the purchase. If purchase is
// in PENDING state then ensure the entitlement is marked as pending and the
// user does not receive benefits yet. It is recommended that this step is
// done on your secure backend and can combine in the API call to your
// backend in step 1.

// Step 3: Notify the user using appropriate messaging.
if (purchase.getPurchaseState() == PurchaseState.PURCHASED) {
    for (String product : purchase.getProducts()) {
        Log.d(TAG, product + " purchased successfully! ");
    }
}

// Step 4: Notify Google the purchase was processed.
// For one-time products, acknowledge the purchase.
// This sample app (client-only) uses billingClient.acknowledgePurchase().
// For consumable one-time products, consume the purchase
// This sample app (client-only) uses billingClient.consumeAsync()
// If you have a secure backend, you must acknowledge purchases on your server using the
// server-side API.
// See https://developer.android.com/google/play/billing/security#acknowledge
if (purchase.getPurchaseState() == PurchaseState.PURCHASED && !purchase.isAcknowledged())
{

    if (shouldConsume(purchase)) {
        ConsumeParams consumeParams =
            ConsumeParams.newBuilder().setPurchaseToken(purchase.getPurchaseToken()).build();
        billingClient.consumeAsync(consumeParams, consumeResponseListener);

    } else {
        AcknowledgePurchaseParams acknowledgePurchaseParams =
            AcknowledgePurchaseParams.newBuilder()
                .setPurchaseToken(purchase.getPurchaseToken())
                .build();
        billingClient.acknowledgePurchase(
            acknowledgePurchaseParams, acknowledgePurchaseResponseListener);
    }
}
}

```

5. 分析購買交易放棄率

到目前為止，程式碼研究室的 Play 帳款服務回應著重於有限的情境，例如 `USER_CANCELLED`、`BILLING_UNAVAILABLE`、`OK` 和 `ITEM_ALREADY_OWNED` 回應。不過，Play 帳款服務可能會傳回 13 種不同的回應代碼，這些回應代碼可能由各種實際因素觸發。

本節將詳細說明 `USER_CANCELLED` 和 `BILLING_UNAVAILABLE` 錯誤回應的原因，並建議您可採取的修正動作。

USER_CANCELED 回應錯誤代碼

這個回應代碼表示使用者在完成購買前，已放棄購買流程 UI。

可能原因	您可以採取哪些行動？
- 可能表示使用者意圖不高，且對價格很敏感。 - 交易待處理或付款遭拒。	- 重新評估區域定價策略，解決意願低的使用者問題。 - 使用購物車棄單功能，提醒使用者完成購物車中待處理的交易。詳情請參閱「設定購買流程建議以提高訂單量」。 - 建議翻譯應用程式並提供本地化內容，吸引更多使用者。 - 為產品提供區域專屬價格和折扣優惠。

BILLING_UNAVAILABLE 回應錯誤代碼

這個回應代碼表示由於使用者的付款服務供應商或所選付款方式發生問題，因此無法完成交易。例如使用者的信用卡已過期，或使用者位於不支援的國家/地區。這項程式碼並不表示 Play 帳款服務本身發生錯誤。

可能原因	您可以採取哪些行動？
- 使用者裝置上的 Play 商店應用程式版本過舊。 - 使用者所在的國家/地區不支援 Google Play。 - 使用者是企業使用者，且該企業的管理員已禁止使用者購買產品。 - Google Play 無法依據使用者的付款方式收取款項。舉例來說，使用者的信用卡可能已過期。	- 監控系統問題和特定區域的趨勢 - 建議改用 <code>PBL 8</code>，因為該版本支援更精細的 <code>PAYMENT_DECLINED_DUE_TO_INSUFFICIENT_FUNDS</code> 子回應代碼。如果收到這個回應代碼，請考慮通知使用者交易失敗，或建議使用其他付款方式。 - 這個回應代碼專為重試而設計，方便您實作合適的重試策略。在這種情況下，自動重試可能無法解決問題。不過，如果使用者能解決造成問題的原因，就能透過手動重試方式解決問題。舉例來說，如果使用者將 Play 商店更新至支援的版本，就能以手動重試方式執行初始作業。 如果錯誤是在使用者非處於工作階段的期間發生，重試可能不太合理。如果您收到因購買流程而產生的 <code>'BILLING_UNAVAILABLE'</code> 回應，很可能是因為使用者在購買過程中收到來自 Google Play 的意見回饋，且可能知道問題為何。在這種情況下，您可以對使用者顯示錯誤訊息，指出系統發生錯誤，並提供「再試一次」按鈕，讓使用者在解決問題後可以手動重試。

回應錯誤代碼的重試策略

針對 Play 帳款服務程式庫 (PBL) 的可復原錯誤，有效的重試策略會因情境而異，例如使用者在工作階段中的互動 (如購買期間) 與背景作業 (如在應用程式恢復時查詢購買交易)。請務必實作這些策略，因為某些 `BillingResponseCode` 值表示暫時性問題，可以透過重試解決，其他值則表示永久性問題，不需要重試。

如果使用者處於工作階段中發生錯誤，建議採用簡易的重試策略，並設定嘗試次數上限，盡量避免干擾使用者體驗。反之，對於背景作業 (例如確認新購買交易) 這類不需要立即執行的作業，建議採用指數輪詢。

如要進一步瞭解特定回應代碼和對應的建議重試策略，請參閱「[處理 BillingResult 回應代碼](#)」。

步驟 6 · 約 1 分鐘

6. 後續步驟

- 瞭解如何[充分發揮 Play 帳款服務整合效益](#)。
- 請務必在使用者開始購買這些產品後，遵循[驗證及處理購買交易的最佳做法](#)，在安全後端進行相關作業。

參考文件

- [單次產品總覽](#)
 - [將 Google Play 帳款服務程式庫整合至應用程式](#)
 - [單次產品的多個購買選項和優惠](#)
-

步驟 7 · 約 1 分鐘

7. 恭喜！

恭喜！您已順利透過 Google Play 管理中心建立新的單次產品、測試帳單結算回應代碼，並分析購買交易中斷情形。

問卷調查

我們非常重視您對本程式碼研究室的意見。請考慮花幾分鐘時間填寫問卷調查。
